

SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

By: Omelette Squad

Members: Raphael Tang, Wong Wei Jun, Leong Yu Hung,
Nataneal Hew, Loh Kok Hao

Table of Contents

1. Introduction	2
2. System Architecture & Design	4
3. Functional Requirements & Scope	12
4. Monitor, Evaluation, Assumptions & Dependencies	15
5. Project Management & Team Contributions	18
6. Conclusion	20

1.0 Introduction

E.G.G (Employment Guidance Generator) is an AI-assisted career guidance platform that helps students and early-career users choose suitable career pathways based on their personal profile and real-world labour market information. The system collects a concise user profile, optionally reads a resume or CV, combines that context with structured career datasets, and calls an OpenAI-compatible API to return ranked career recommendations in a structured JSON format.

Unlike a static career quiz, E.G.G is designed as a decision intelligence system. It weighs personal fit, salary return, education cost, risk tolerance, market demand, automation risk and graduate outcomes before presenting recommendations. This makes the system more useful for users who need practical, financially realistic guidance rather than generic motivational advice.

1.1 Purpose

The purpose of this System Analysis Documentation is to describe the technical scope, architecture, system design decisions, data movement, LLM integration, MVP features, quality requirements, monitoring assumptions, dependencies and project management plan for E.G.G. The document follows a hackathon-oriented format similar to the provided UMHackathon SAD sample, where the focus is on high-impact technical feasibility rather than lengthy traditional SRS narration.

The key system elements covered in this document are as follows:

- **Architecture:** The platform is structured as a client-server web application with a Next.js frontend and a FastAPI backend.
- **Data Flows:** User profile and optional uploaded document data move from the browser to the backend, then into a prompt-context layer before being sent to the LLM API.
- **Model Process:** The LLM service performs reasoning using user context, dataset evidence and a fixed JSON response schema.
- **Role of Reference:** This document supports developers, testers and evaluators in understanding how the system is expected to work.

1.2 Background

Students often struggle to choose a career because the decision involves many competing factors: personal interests, course cost, skills, graduate employability, salary prospects, industry demand and risk of underemployment. Traditional career counselling depends heavily on general advice and may not include updated market datasets. E.G.G addresses this problem by combining structured datasets with AI reasoning so that the recommendation is both personalized and evidence-informed.

The existing prototype contains a Next.js frontend with form-based profile collection, a Python FastAPI backend with a POST /analyze endpoint, OpenAI-compatible API integration, PDF/DOCX reading support, and multiple JSON/CSV datasets related to careers, skills, market analysis, graduate statistics, workforce indicators and course information. The platform is therefore suitable for a hackathon MVP because it demonstrates end-to-end functionality from user input to AI-generated recommendation.

Major capabilities introduced in this version include resume/CV upload, AI-generated structured JSON output, career ROI calculation, break-even estimation, financial risk tagging and market-reality explanations.

1.3 Target Stakeholders

Stakeholder	Roles	Expectations
Student / Job Seeker	Enters education background, skills, preferred tasks, work environment and salary/location expectations. May upload a resume or CV.	Simple career guidance flow, realistic recommendations, clear explanation of trade-offs and next steps.
Career Advisor / Mentor	Uses system output to support counselling discussion and compare career pathways.	Transparent reasoning, evidence-based recommendation factors, and outputs that are easy to explain to users.
Development Team	Builds and maintains frontend, backend, dataset ingestion, prompt logic and API integration.	Clear service boundaries, modular codebase, documented API request/response format.
QA Team	Validates input handling, API behaviour, response parsing, errors and result presentation.	Defined scope, edge-case coverage and testable acceptance criteria.
Hackathon Judges	Evaluate innovation, implementation quality, feasibility and impact.	Strong technical narrative, clear architecture and meaningful AI integration.

2.0 System Architecture & Design

2.1 High Level Architecture

Type	Details
System	Web-based AI career recommendation platform.
Frontend	Next.js 15 application using React components such as UserForm and ResultsPanel.
Backend	Python FastAPI application exposing /analyze for profile submission and analysis.
AI Layer	OpenAI-compatible chat completion API using a structured prompt and JSON-only response contract.
Data Layer	Local JSON and CSV datasets including career descriptions, graduate statistics, market analysis, in-demand jobs, skill mappings, course information and workforce data.
Deployment	Dockerfiles exist for frontend and backend, with docker-compose available for local multi-service deployment.

E.G.G follows a client-server architecture. The frontend captures user input and sends it as FormData to the backend. The backend reads optional resume documents, loads datasets, constructs a long-form reasoning prompt and sends the prompt to the OpenAI-compatible API. The response is parsed as JSON and returned to the frontend, where results are shown as ranked career cards.

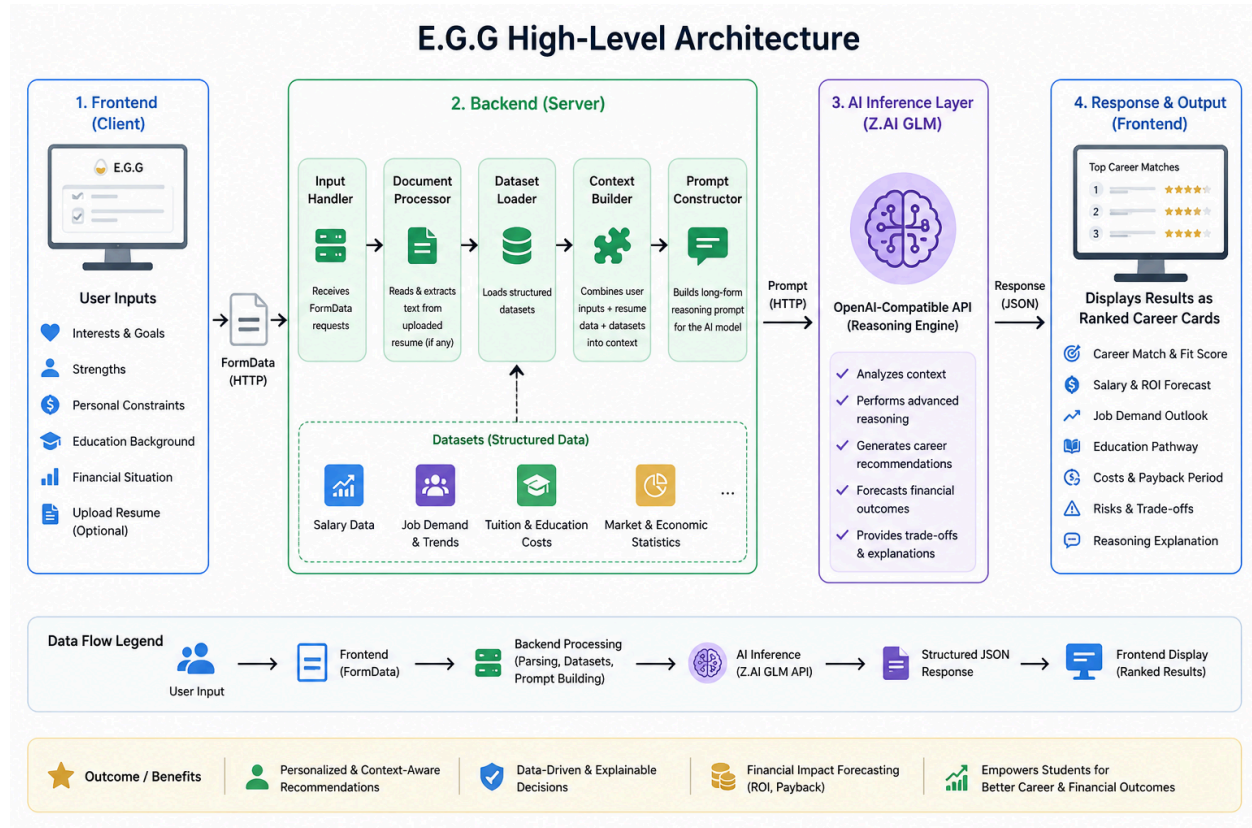


Figure 1: High-Level Architecture of E.G.G

2.1.2 LLM as Service Layer

The LLM is not treated as a vague “AI Module”. In E.G.G, it is implemented as a service layer that receives a constructed context window containing user profile information, extracted resume text and selected dataset summaries. The backend controls the prompt contract, defines the scoring formulas, specifies policy rules, enforces a JSON-only output structure and validates the response through JSON parsing before sending it to the frontend.

The prompt includes dataset evidence such as workforce intelligence, market analysis, course fee information, graduate statistics, future skills mapping, in-demand job signals, career path descriptions and job meta information. This makes the LLM output more grounded than a normal chatbot answer because it is pushed to reason within the boundaries of data supplied by the system.

The backend also truncates uploaded document content to approximately 3000 characters before injecting it into the prompt. This is important for token control and cost management. In a production version, further chunking and retrieval would be added so that only the most relevant sections of the CV are included.

The backend is responsible for all numerical computations including ROE, break-even years, and final weighted scoring. The LLM is strictly used for interpretation, ranking explanation, and

generating user-friendly career narratives based on precomputed values. The LLM is not allowed to modify or recompute numerical outputs.

Figure 2: LLM-as-Service Dependency Diagram

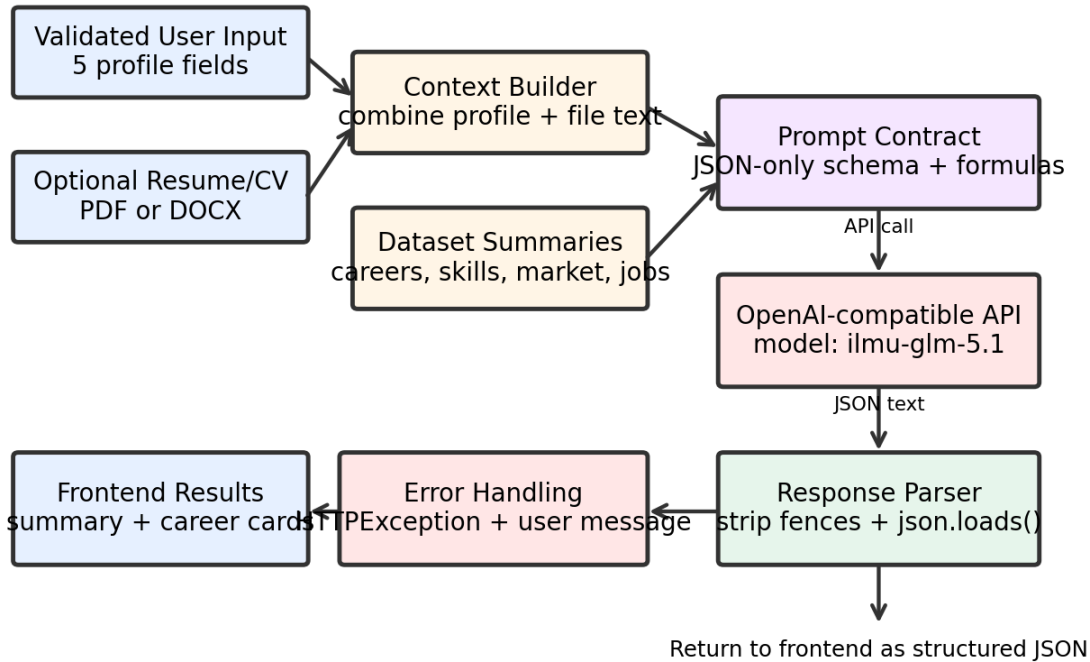


Figure 2: LLM-as-Service Dependency Diagram

Scoring Logic:

The system applies structured evaluation metrics within the prompt:

- Return on Education (ROE):

$$\text{ROE} = (\text{Estimated 5-Year Earnings} - \text{Education Cost}) / \text{Education Cost}$$

- Break-Even Years:

Time required to recover education cost from salary

- Final Score:

Weighted combination of:

- Personal Fit (skills, interests)
- Market Demand
- Financial Return (ROE)
- Risk Factors (automation, volatility)

2.1.3 Dependency Diagram Explanation

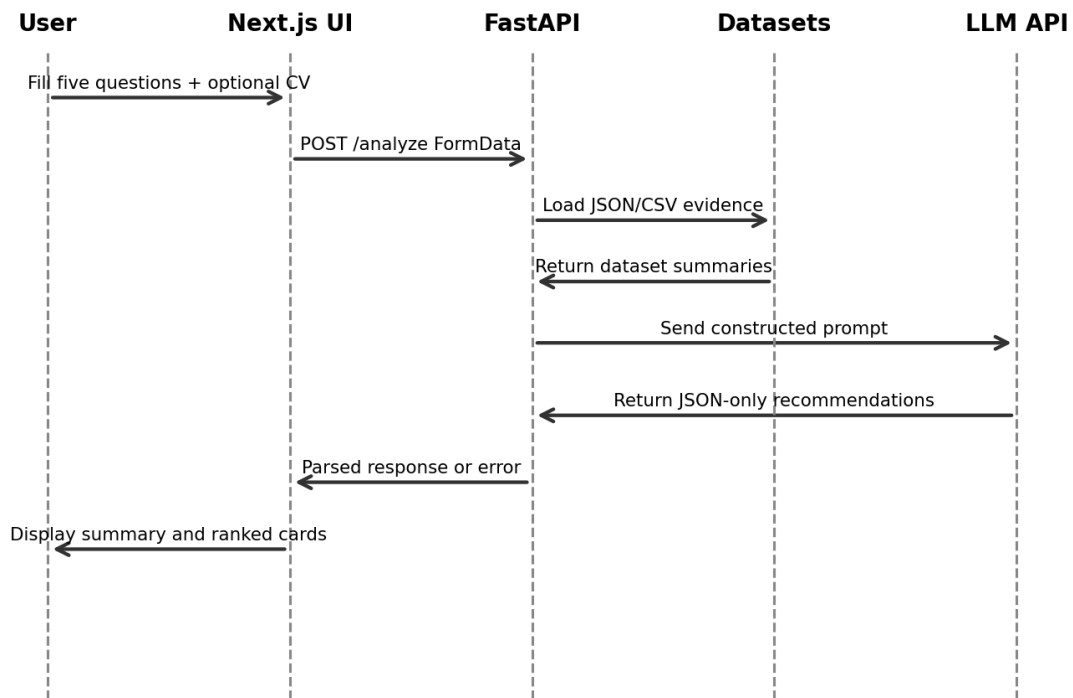
The dependency flow begins with the user completing five profile questions. If a PDF or DOCX file is uploaded, the backend extracts readable text and appends it as additional context. The backend then loads multiple JSON/CSV datasets and builds a prompt containing formulas for Annual Salary, 5-Year Earnings, Return on Education, Break-Even Years, Financial Score and Final Score.

The API call is made using an OpenAI-compatible client configuration. The implementation uses a base URL pointing to an external OpenAI-compatible provider and requests a model named ilmu-glm-5.1. Although the project describes the AI layer as OpenAI/API-based, the code is designed around the OpenAI client interface, which allows the backend to swap between compatible providers with limited changes.

Component	Input	Output	Failure Risk	Mitigation
UserForm	Five required text fields and optional file	FormData request	Missing or invalid user input	Frontend validation before submission.
FastAPI /analyze	description, timestamp, budget, risk, optional file	Structured career analysis JSON	Server error, invalid file, CORS issue	HTTP error handling and user-facing retry message.
Dataset Loader	JSON/CSV files	Python dictionaries and records	Missing file or invalid CSV	Startup logging and fallback empty structures.
LLM API	Prompt context and response schema	JSON text containing recommendations	Rate limit, timeout, invalid JSON	Timeout, parsing validation, graceful error.
ResultsPanel	Parsed JSON results	Career cards and summary	Unexpected schema	No-results fallback display.

2.2.2 Sequence Diagram

The primary user journey is “Generate Career Recommendation”. The flow begins when the user fills in career profile details and optionally uploads a resume. The frontend submits this information to the backend. The backend validates the request, reads uploaded documents, loads datasets, constructs the prompt, calls the LLM API, parses the result and returns a clean JSON response to the frontend.

Figure 3: Sequence Diagram - Career Recommendation Flow*Figure 3: Sequence Diagram - Career Recommendation Flow*

This journey demonstrates the core technical value of the system: the user does not directly interact with the LLM. Instead, the backend acts as a controlled reasoning layer that defines what context is sent, what formulas must be used, how the output must be structured and how the frontend should receive the data.

2.2 Technological Stack

Layer	Technology	Justification
Frontend	Next.js 15, React 18, CSS modules/files	Provides a fast web interface with component-based UI and simple local deployment.
Backend	Python FastAPI	Suitable for lightweight REST endpoints, async request handling and easy AI API integration.
AI/API	OpenAI-compatible Chat Completions API	Allows prompt-based reasoning and structured JSON responses for career recommendation.
Data	JSON and CSV datasets	Simple to inspect, update and demo during hackathon without heavy database setup.
Document Parsing	PDF/DOCX reader functions	Allows resume/CV extraction to improve context-aware recommendation.
Deployment	Docker and docker-compose	Supports repeatable local environment for frontend and backend.

The technology stack is intentionally practical for a hackathon MVP. Next.js keeps the frontend simple, FastAPI keeps the backend flexible, and JSON/CSV datasets avoid unnecessary database complexity during the prototype stage. This allows the team to focus on the core AI-assisted recommendation flow rather than spending time building administrative infrastructure.

For a future production version, the local datasets could be migrated to PostgreSQL or a vector database. Authentication and audit logs should also be added if the system stores user profiles or resumes permanently.

2.3 Key Data Flows

The key data flow in E.G.G is centred on transforming raw user input into explainable career recommendations. The system does not simply send the user text to the AI. Instead, it enriches that text with datasets and requires the LLM to return a fixed JSON response. This prevents the frontend from depending on unstructured chatbot text.

Figure 4: Data Flow Diagram (Level 0)

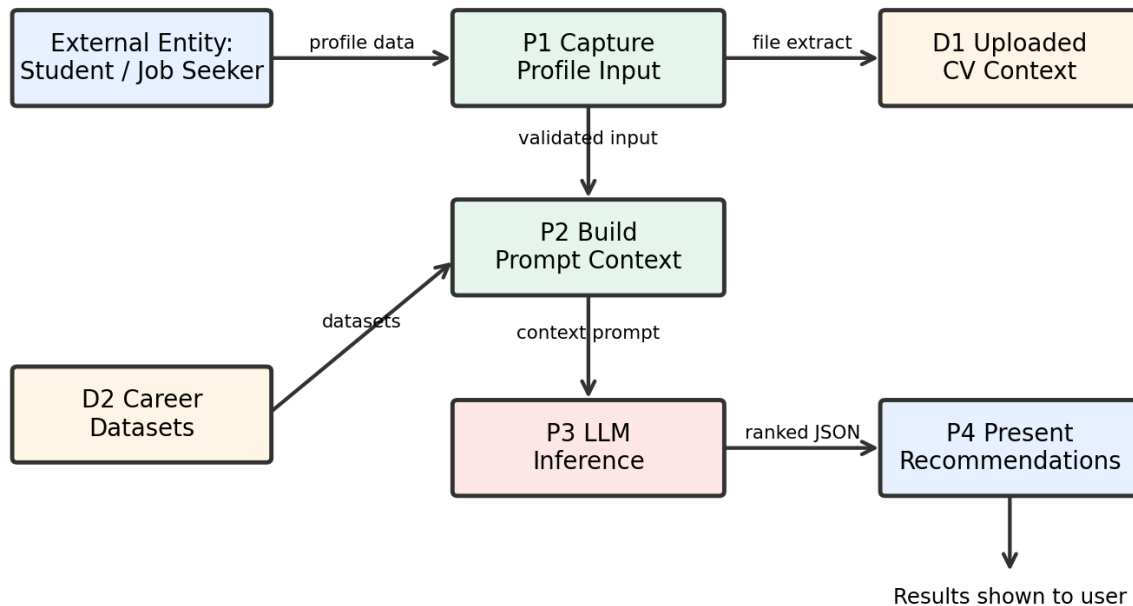


Figure 4: Data Flow Diagram (Level 0)

The main data objects are profile description, timestamp, budget constraint, risk tolerance, optional file context, dataset summaries, constructed prompt, LLM JSON response and rendered result cards. Sensitive data such as CV contents should be processed only for the active analysis session and should not be stored unless explicit consent is introduced in a future version.

Security Considerations:

- Uploaded CVs are processed in-memory and not permanently stored.
- API keys are stored securely in environment variables.
- File uploads are restricted to PDF/DOCX and validated before processing.
- Future versions should include rate limiting and authentication to prevent abuse.

2.3.2 Normalized Database Schema / Logical Data Model

The current MVP mainly uses local JSON and CSV files rather than a relational database. However, the data can still be represented logically using normalized entities. A future database design should separate user profile, career, market signal, course information, skill mapping and recommendation result. This separation reduces duplication and allows different sources to be updated independently.

Figure 5: Logical Data Model / Normalized Dataset View

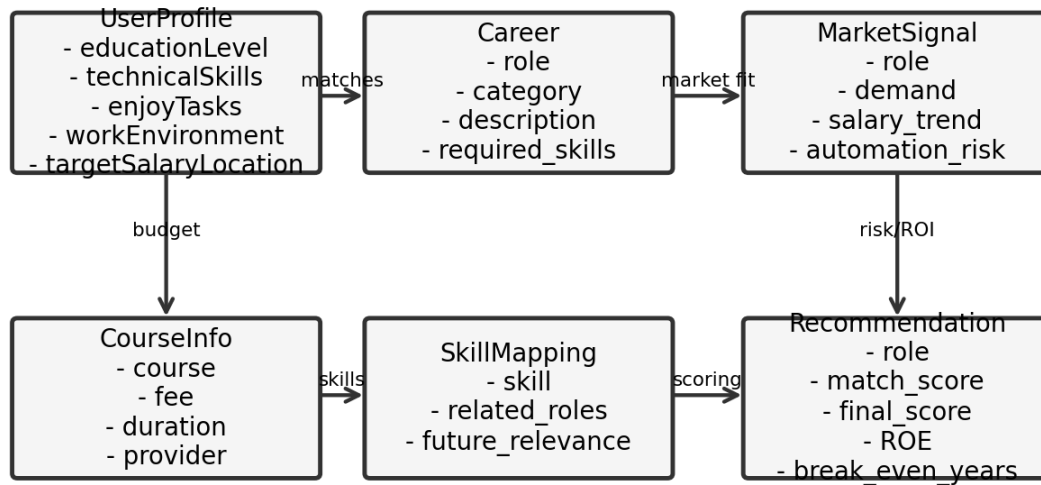


Figure 5: Logical Data Model / Normalized Dataset View

Entity	Purpose	Example Attributes
UserProfile	Temporary representation of the user's career background and preferences.	educationLevel, technicalSkills, enjoyTasks, workEnvironment, targetSalaryLocation
Career	Stores career role and descriptive information.	role, category, required_skills, short_description
MarketSignal	Stores market demand and risk information.	demand_level, salary_growth_rate, automation_risk

CourseInfo	Stores education cost and duration data.	course_name, local_fee_per_year, duration
Recommendation	Stores generated output if persistence is later required.	match_score, final_score, roe_percentage, break_even_years

3.0 Functional Requirements & Scope

The functional scope focuses on a high-impact MVP that can be demonstrated within the hackathon period. The goal is not to build a full employment platform, but to prove the feasibility of AI-supported career recommendation using user context and real-life data.

3.1 Minimum Viable Product

#	Feature	Description
1	Profile Input Form	Users answer five questions covering education, skills, preferred tasks, ideal work environment and salary/location target.
2	Resume / CV Upload	Users may upload PDF or DOCX documents for additional context extraction.
3	AI Career Analysis	Backend constructs a structured prompt and sends it to an OpenAI-compatible API.
4	Dataset-Cross-Referenced Recommendation	The recommendation uses labour market, skill mapping, course cost, graduate outcome and job demand datasets.
5	Result Display	Frontend displays a summary, ranked career cards, match score, explanation, trade-offs, next steps and market/financial insights.

The MVP is strong because every feature directly supports the main demo story: “A user provides personal context, the system cross-references market data, and the AI returns explainable career options.”

3.1.1 Functional Requirements

Requirement ID	Requirement	Priority	Acceptance Criteria
FR-001	The system shall collect all required user profile fields before submission.	High	Submission is blocked if any required field is empty.
FR-002	The system shall accept optional PDF or DOCX files only.	High	Other file types are rejected with a clear message.
FR-003	The system shall send profile and file data to the backend using FormData.	High	Backend receives description, timestamp and file object if provided.
FR-004	The backend shall extract text from supported uploaded documents.	Medium	Extracted text is appended as additional prompt context.
FR-005	The backend shall load career, skills, workforce, market and course datasets.	High	Dataset summaries are available to the prompt builder.
FR-006	The backend shall call an OpenAI-compatible API for structured reasoning.	High	A JSON-only response is requested using a fixed schema.
FR-007	The system shall parse and return AI output as valid JSON.	High	Invalid JSON causes a controlled server error.
FR-008	The frontend shall display ranked career recommendations.	High	Cards show role, match score, explanations and next steps.

The functional requirements are written to be testable. Each requirement describes a visible behaviour or backend process that can be verified through manual testing, API testing or code inspection.

3.2 Non-Functional Requirements (NFRs)

Quality	Requirement	Implementation
Scalability	System should support more datasets and more career	Keep dataset loading modular and migrate to

	roles without redesigning the user flow.	database/vector retrieval in later versions.
Reliability	Career analysis should not crash the frontend if the backend or API fails.	Use frontend error state and backend exception handling.
Maintainability	Prompt rules, formulas and datasets should be changeable without rewriting UI components.	Separate UserForm, ResultsPanel, backend prompt construction and dataset loader.
Token Latency	AI response should normally return within an acceptable demo time window.	Use timeout settings, concise context, file text truncation and loading state.
Cost Efficiency	Average token usage per session should be controlled.	Limit file extract length, avoid repeated huge dataset injection, cache common dataset summaries in future.
Security	API keys and user documents must be protected.	Store keys in environment variables and avoid permanent CV storage in MVP.
Usability	A new user should understand the career input flow without instructions.	Use five guided questions, placeholders, hints and clear result cards.

The most important NFRs for this project are reliability, token latency and cost efficiency because the core feature depends on an external LLM API. If the AI call fails or becomes too slow, the value of the product is immediately affected.

3.3 Out of Scope / Future Enhancements

The following features are intentionally excluded from the MVP to keep the project focused and achievable within the hackathon timeline:

- User authentication and account history.
- Permanent storage of uploaded resumes and previous recommendations.
- Real-time job vacancy scraping or direct job application submission.
- In-app chat with a career counsellor.
- Payment, subscription or premium user management.
- Full admin dashboard for editing datasets through the browser.
- Production-level analytics dashboard and A/B testing platform.

Future enhancements may include live job market integration, salary prediction by location, vector search over career descriptions, authenticated user profiles, saved recommendation history, multilingual support and a mentor dashboard. These are valuable but should not distract from the MVP demonstration of AI-assisted career decision intelligence.

4.0 Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

The technical evaluation strategy focuses on whether E.G.G produces useful, consistent and technically reliable results. Evaluation should consider both software behaviour and recommendation quality. Since the system uses an LLM, evaluation must include schema validity, response latency, explanation quality and whether recommendations align with supplied dataset evidence.

4.1.1 Grayscale Rollout & A/B Testing

For a production deployment, E.G.G should be released using a grayscale rollout. For example, a new scoring formula or prompt version may first be exposed to 5% of users. If JSON validity, latency and user satisfaction remain stable, rollout can gradually increase to 25%, 50% and 100%. A/B testing can compare two prompt templates: one emphasizing personal fit and one emphasizing financial ROI.

4.1.2 Emergency Rollback & Golden Release

A golden release should be maintained as the last stable backend and prompt version. If the system begins returning invalid JSON, irrelevant careers or slow responses, the team can rollback to the golden release. This is especially important because a small prompt change can significantly affect model output quality.

4.1.3 Priority Matrix

Priority	Trigger Condition	Action
P1 Critical	AI response failure above 10% or JSON parse failure above 5%.	Rollback prompt/API version, disable file upload if needed, show graceful message.
P2 High	Average response time exceeds 15 seconds during demo.	Reduce prompt length, trim datasets, cache summaries, increase timeout if safe.
P3 Medium	Recommendation explanations are too generic.	Revise prompt to cite dataset factors and require trade-offs.
P4 Low	UI display issue but backend works.	Patch frontend styling after demo-critical functions are stable.

4.2 Monitoring Plan

Monitoring should cover frontend availability, backend API health, LLM response time, JSON parse success rate, error rate, file upload failures and user completion rate. In the MVP, these can be monitored through console logs and manual testing. In production, the system should use structured logs and dashboards.

Metric	Purpose	Suggested Alert
/analyze success rate	Measures core feature reliability.	Alert if success rate drops below 95%.
LLM latency	Measures response speed and user experience.	Alert if p95 latency exceeds 15 seconds.
JSON parse failure	Detects prompt or model response issues.	Alert if failures exceed 5%.
File extraction errors	Detects unsupported or unreadable documents.	Alert if errors rise suddenly.
Frontend abandonment	Measures whether users complete the form.	Review UI if many users stop before submission.

4.3 Assumptions

- Users have stable internet access during analysis.
- The OpenAI-compatible API key is valid and correctly configured in the environment.
- Dataset files are available and readable when the backend starts.
- Users provide honest and meaningful profile answers.
- Uploaded resumes contain readable text and are not scanned images requiring OCR.

4.4 External Dependencies

Tool / Service	Purpose	Risk Level	Mitigation
OpenAI-compatible LLM API	Core reasoning engine for career recommendation and JSON output.	High	Add timeout, fallback message, cache common responses and keep golden prompt.

PDF/DOCX Parsing Libraries	Extract resume text for better context.	Medium	Validate file type and limit extracted text length.
JSON/CSV Datasets	Ground recommendations with career and market evidence.	Medium	Validate dataset format and maintain backups.
Docker	Local service packaging and deployment consistency.	Low	Document manual setup as fallback.

4.5 Test Strategy

Testing for E.G.G should cover both traditional software behavior and AI-specific behavior. Traditional testing verifies that inputs, file uploads, API calls and UI rendering work correctly. AI-specific testing verifies that the returned JSON schema is valid, recommendations are relevant, and financial fields such as ROE and break-even years are present.

Test ID	Scenario	Steps	Expected Result
TC-001	Valid profile without file	Complete all five fields and submit.	System returns ranked career recommendations.
TC-002	Empty required field	Leave one required field empty and submit.	Frontend blocks submission and shows alert.
TC-003	Unsupported file upload	Upload a .txt or image file.	System rejects file and requests PDF/DOCX.
TC-004	PDF/DOCX upload	Upload readable resume and submit.	Backend extracts text and includes it in analysis context.
TC-005	Backend unavailable	Stop backend and submit form.	Frontend shows clear connection or server error.
TC-006	Invalid AI JSON	Simulate malformed model response.	Backend raises controlled error instead of crashing silently.
TC-007	Results rendering	Return JSON with multiple careers.	ResultsPanel displays summary and career cards correctly.

5.0 Project Management & Team Contributions

5.1 Project Timeline

Day	Activity	Deliverable
Day 1	Problem framing and feature prioritization.	Confirmed E.G.G concept and MVP scope.
Day 2	Dataset collection and cleaning.	Career, workforce, market, skills and course datasets prepared.
Day 3	Frontend form and basic UI.	UserForm page with five profile fields.
Day 4	Backend /analyze endpoint and CORS setup.	FastAPI service receives FormData.
Day 5	LLM prompt construction and JSON schema.	Structured AI response prototype.
Day 6	ResultsPanel and career card display.	Ranked recommendations shown in UI.
Day 7	File upload and PDF/DOCX extraction.	Optional CV context added.
Day 8	Testing and error handling.	Validation, error states and retry flow.
Day 9	Documentation and diagrams.	SAD and architecture diagrams completed.
Day 10	Demo preparation and final polish.	Stable golden build and presentation script.

5.2 Team Member Roles

Role	Responsibility
Raphael Tang	Coordinates scope, timeline, demo story and documentation consistency.
Wong Jun Wei	Builds Next.js UI, form validation, loading/error states and results display.
Loh Kok Hao	Builds FastAPI endpoint, file handling, dataset loading and AI API call.
Nataneal Hew	Prepares JSON/CSV datasets and ensures fields support recommendation logic.

Leong Yu Hang	Designs prompt contract, scoring instructions and JSON response format.
---------------	---

5.3 Recommendations

- Move large datasets into a database or retrieval layer so the prompt does not need to include excessive raw data.
- Add caching for repeated or similar profile queries to reduce token cost and improve response time.
- Add authentication only after the core recommendation flow is stable.
- Store API keys only in environment variables and never expose them in the frontend.
- Create a formal prompt versioning system so the team can rollback unstable prompt changes.
- Add benchmark profiles for testing recommendation consistency before demos.

6.0 Conclusion

E.G.G demonstrates a practical and technically meaningful use of AI for career guidance. Its value comes from combining personal user input, optional resume context, structured career datasets and LLM-based reasoning into a single decision-support workflow. The system is not merely a chatbot; it uses the backend as a controlled service layer that constructs prompts, injects relevant data, enforces formulas and parses structured JSON results.

From a system analysis perspective, the project has a clear problem statement, realistic stakeholders, well-defined MVP features, an explainable architecture, identifiable data flows, measurable non-functional requirements and a manageable deployment plan. The recommended next step is to improve production readiness through dataset retrieval, monitoring, prompt versioning, caching and stronger privacy controls.

Overall, E.G.G is suitable for a distinction-level hackathon submission because it demonstrates innovation, implementation feasibility, AI integration, user impact and a clear plan for future scalability.